

Advanced Programming Techniques

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

Server

fbeit

FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Counterpart of client

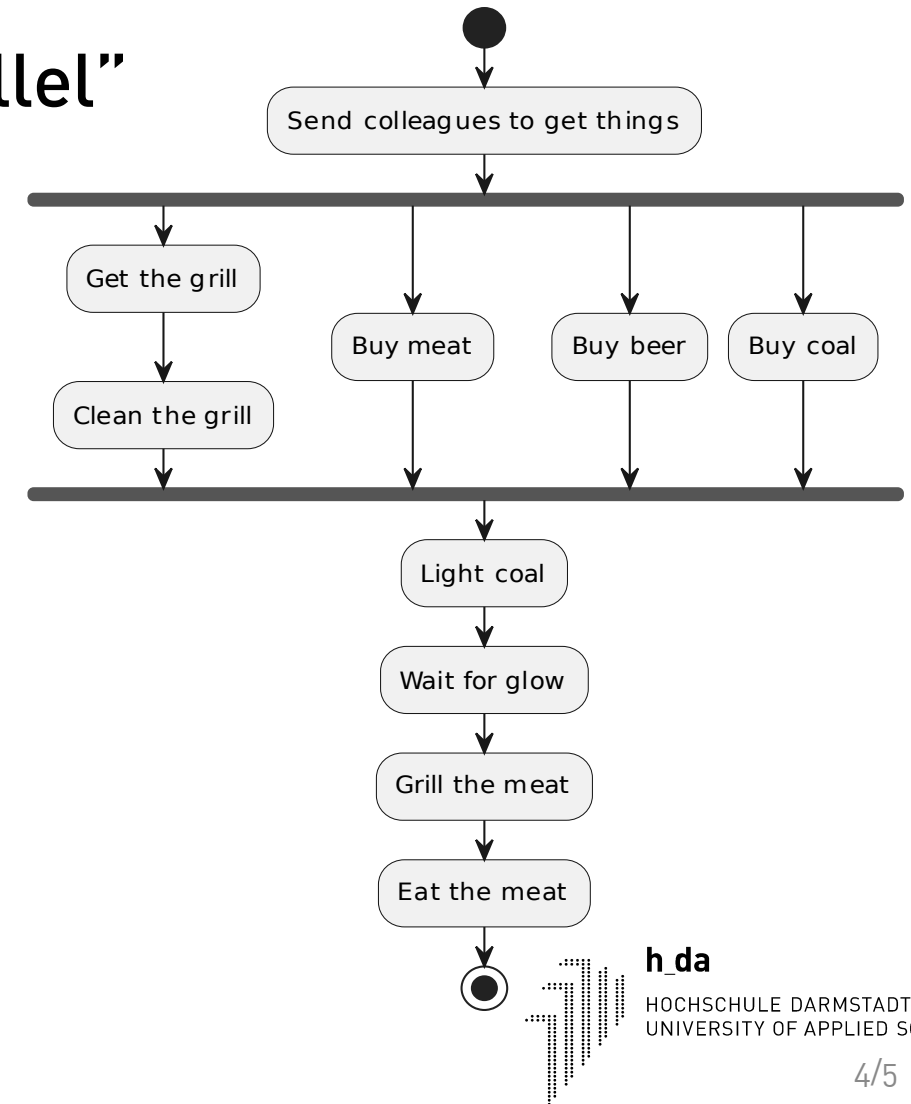
- **Using the PoCo library**
 - Create `ServerSocket`
 - Wait for `StreamSocket` (endpoint of connection to requesting client)
 - (Optional: create `SocketStream`)
- **How about several client in parallel?**

Live Coding

Tasks

- Sometimes, things can be done “in parallel”
- In programs, things are done (code is executed) by “tasks”
 - Up to now, we had only a single tasks
- **Tasks**
 - Are created by “forking”
 - Work independent of each other
 - You can wait for the completion of a task by “joining”

Live Coding



Task synchronization

- Sometimes, tasks require exclusive access to resources
 - E.g. task 2 has to wait until task 1 has finished using the resource
- Access can be controlled using a **“mutex”**
 - Mutex = “mutual exclusion”
 - Before executing some section of code, a task locks the mutex
 - After the execution, the mutex is unlocked
 - While locked, another task attempting to lock the mutex is suspended
- Enforce usage pattern with a **“lock_guard”**

Live Coding